

Informe de Avance y Requisitos: Vehículo Autónomo Navix

Juan Pablo Santiago Castaño Loaiza

5 de junio de 2026

1. Novedades del Equipo de Trabajo

Para dar cumplimiento a los requerimientos administrativos del proyecto, se informa que **no se presentan novedades** en la conformación del equipo de trabajo, ni cambios en el nombre del proyecto respecto a entregas anteriores. El equipo sigue conformado por Juan Pablo y Santiago Castaño Loaiza, y el proyecto mantiene el nombre de **Navix**.

2. Tareas y Porcentaje de Avance

El proyecto se encuentra actualmente en un **50 % de desarrollo global**. Las tareas se han distribuido de la siguiente manera:

- **Juan Pablo:** Tarea prioritaria en la finalización de las pruebas unitarias de los subsistemas de hardware (Motor, Radar, Servo).
- **Santiago Castaño Loaiza:** Investigación del uso del segundo núcleo (Multicore) del RP2040, asegurando la correcta sincronización según el SDK de Raspberry Pi, y la nueva funcionalidad del proyecto de coordenadas GPS.

El estado actual de las pruebas es el siguiente:

- Prueba unitaria del Radar HC-SR04: 100 %
- Prueba unitaria del Servo SG90: 100 %
- Prueba unitaria de Motores (TB6612FNG): 100 %
- Prueba de integración Servo-Radar: 100 %
- Prueba unitaria del GPS: 20 %
- Prueba integración GPS-Motores: 0 %
- Lógica de navegación principal: En desarrollo.

3. Cambios en el Proyecto y Nuevos Requisitos

Respecto a la definición inicial del proyecto, se ha incorporado la integración de un **módulo GPS**. Este requisito añadido tiene como fin el envío de coordenadas a la RP2040 y la posterior navegación autónoma del carro hasta dichas coordenadas evitando los obstáculos.

Aclaración de alcance: Se menciona explícitamente que la integración de módulos de cámara, algoritmos de inteligencia artificial avanzados y reconocimiento de voz **quedan estrictamente fuera del alcance funcional** de la entrega final actual, considerándose únicamente como futuras mejoras teóricas.

4. Requisitos Funcionales: Maniobra de Contingencia

Se corrige y completa la definición del requisito funcional crítico ante un encierro total (callejón sin salida o callejón en U).

- **Lógica de contingencia:** Si el vehículo detecta un obstáculo frontal a menos del umbral de seguridad (20 cm), el servomotor realizará un barrido. Si *ambos flancos* (izquierdo y derecho) presentan distancias inferiores al umbral de seguridad, el sistema no tiene un estado indefinido. El vehículo detendrá sus motores, ejecutará una maniobra de retroceso en línea recta durante 2 segundos (o la distancia equivalente a 30 cm) y volverá a hacer el barrido de obstáculos hasta que detecte un posible escape del callejón.

5. Requisitos No Funcionales Ajustados

Con base en los límites físicos de los actuadores y los tiempos de propagación de las ondas acústicas, se han ajustado los siguientes requisitos para garantizar su viabilidad técnica:

- **Latencia de reacción:** Se relaja el requisito temporal de 100 ms. El nuevo tiempo máximo garantizado para procesar una lectura y ajustar los motores será de **500 ms**, tomando en cuenta el tiempo de barrido físico del servomotor SG90 y la propagación del sonido en el HC-SR04.
- **Concurrencia:** Para la comunicación multicore, no se implementará una arquitectura “lock-free” desde cero. En su lugar, el equipo se compromete a utilizar las primitivas de concurrencia estándar (Mutexes y Spinlocks) proporcionadas de forma nativa por el SDK del RP2040 para evitar condiciones de carrera.

6. Arquitectura de Software

El firmware de Navix se ejecuta en una Raspberry Pi Pico (RP2040). La lógica aprovecha el estado de bajo consumo (`_wfi`) para optimizar el ciclo de ejecución.

Figura 1: Prueba de Motores — Main (polling) e ISR del timer

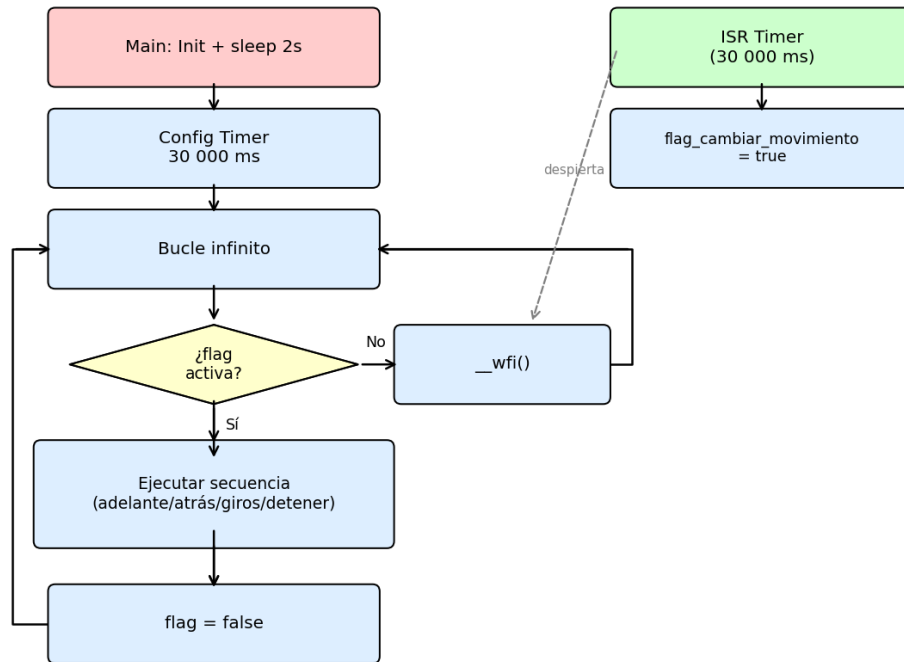


Figura 1: Prueba unitaria de motores. El `main` inicializa el sistema, espera 2 segundos y configura un timer de 30 000 ms; luego entra en un bucle infinito donde, si la bandera `flag_cambiar_movimiento` es falsa, el procesador duerme con `__wfi()`. La ISR del timer se activa cada 30 segundos y pone la bandera en `true`, despertando al procesador. Cuando la bandera es verdadera, el `main` ejecuta secuencialmente la rutina bloqueante (adelante, atrás, giro derecha, giro izquierda, detener) con pausas de `sleep_ms(2000)` entre cada movimiento, baja la bandera y vuelve a dormir.

Figura 2: Prueba del Servo — Main duerme; toda la lógica en la ISR del timer

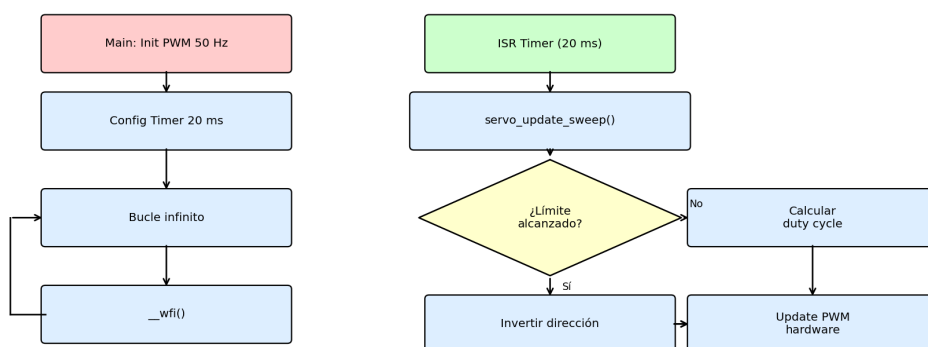


Figura 2: Prueba unitaria del servomotor. El `main` se limita a inicializar el PWM a 50 Hz, configurar un timer periódico de 20 ms y entrar en un bucle infinito que únicamente ejecuta `__wfi()`; el núcleo principal no realiza ningún trabajo de control. Toda la lógica reside en la ISR del timer: cada 20 ms se llama a `servo_update_sweep()`, que evalúa si el ángulo actual alcanzó los límites de barrido (10° y 170°); si es así, invierte la dirección; en cualquier caso, calcula el duty cycle correspondiente y actualiza directamente el hardware PWM.

Figura 3: Prueba del Radar — Main (polling), ISR Timer (TRIG) e ISR GPIO (ECHO)

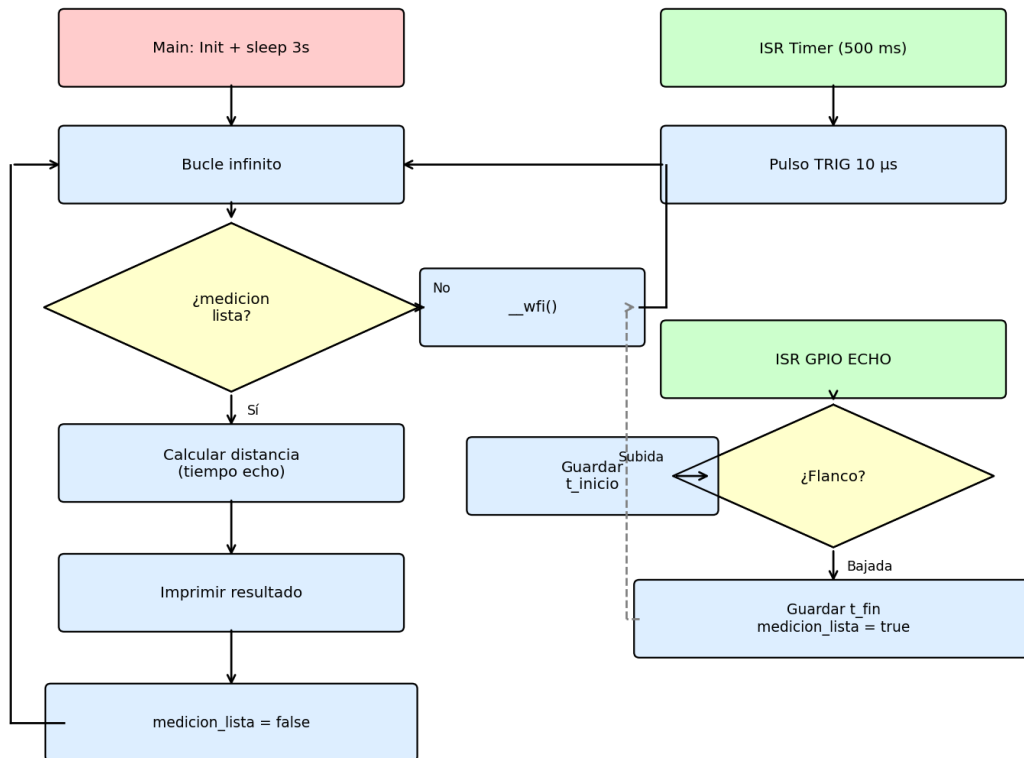


Figura 3: Prueba unitaria del radar ultrasónico HC-SR04. El sistema cuenta con dos interrupciones independientes: la ISR del timer (cada 500 ms) dispara el pulso **TRIG** de 10 μs para iniciar una medición; la ISR del GPIO **ECHO** captura el flanco de subida (guarda `t_inicio`) y el flanco de bajada (guarda `t_fin` y levanta la bandera `medicion_lista`). El **main**, tras inicializar y esperar 3 segundos, entra en un bucle donde consulta la bandera mediante `radar_esta_listo()` —que desactiva las IRQ temporalmente para una lectura segura—; si hay dato disponible, calcula la distancia a partir del tiempo de eco e imprime el resultado; si no, vuelve a dormir con `_wfi()`.

7. Hardware y Presupuesto

7.1. Diagrama de Bloques Físico

Figura 4: Diagrama de bloques de hardware — dominios de potencia y señales

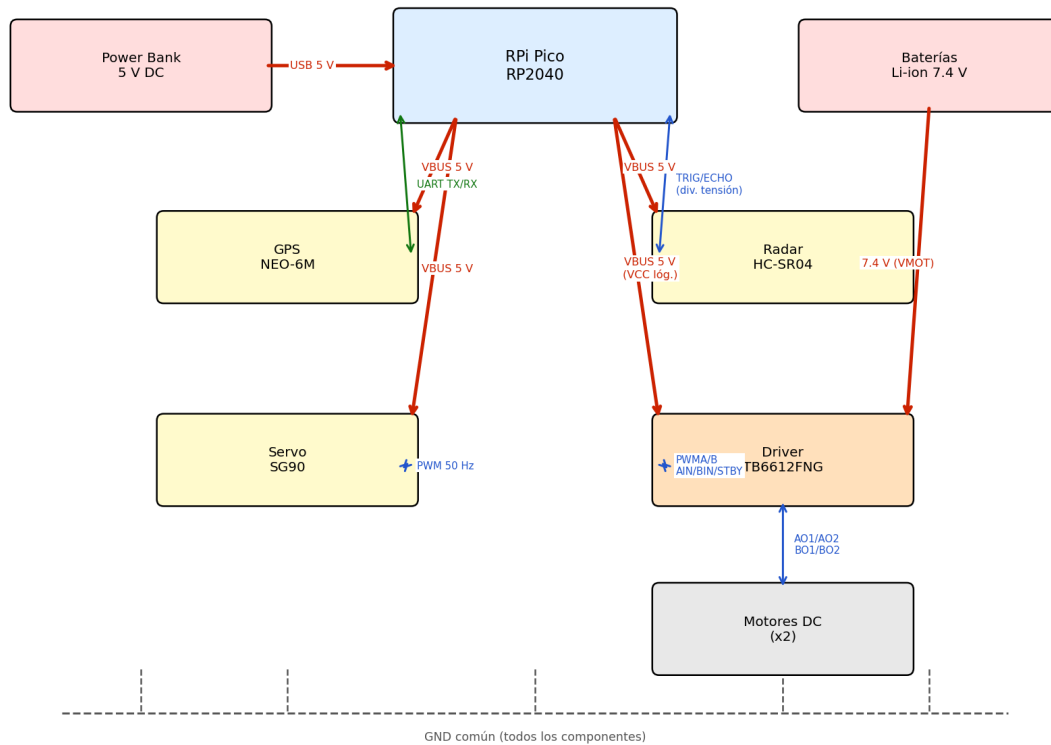


Figura 4: Topología de potencia y señales del sistema Navix. **Dominio de alimentación:** la Power Bank (5 V) se conecta al puerto Micro-USB de la RPi Pico; el pin **VBUS** distribuye esos 5 V como **VCC** al Radar HC-SR04, al módulo GPS NEO-6M, al Servomotor SG90 y al pin **VCC** lógico del driver TB6612FNG. Las baterías Li-ion (7.4 V) se conectan exclusivamente al pin **VMOT** del driver para alimentar los motores DC. Todos los componentes comparten una tierra común (GND). **Dominio de señales:** el GPS se comunica por UART (TX↔RX); el pin **TRIG** del radar va a un GPIO de salida digital y el pin **ECHO** (5 V) pasa por un divisor de tensión a 3.3 V antes de entrar como GPIO de entrada con interrupción; el servo recibe su señal de control por PWM a 50 Hz; el driver recibe las señales **PWMA/B**, **AIN1/2**, **BIN1/2** y **STBY** desde GPIOs de la Pico, y sus salidas **A01/2** y **B01/2** van directamente a los bornes de los motores DC.

7.2. Presupuesto y Costos

Componente	Cantidad	Costo Est. (COP)
Raspberry Pi Pico (RP2040)	1	\$ 25,000
Driver de Motores TB6612FNG	1	\$ 15,000
Motorreductores DC	2	\$ 20,000
Chasis de acrílico y ruedas	1	\$ 30,000
Sensor Ultrasónico HC-SR04	1	\$ 8,000
Servomotor SG90	1	\$ 10,000
Módulo GPS (NEO-6M)	1	\$ 35,000
Power Bank (5 V)	1	\$ 40,000
Baterías Li-ion y accesorios	1 kit	\$ 30,000
Componentes pasivos	Lote	\$ 15,000
Costo Total Estimado		\$ 228,000

Cuadro 1: Presupuesto de ingeniería.

8. Escenario de Pruebas

Para evitar fallas de detección debido a la absorción acústica o desviación de ondas, el entorno de pruebas queda definido con las siguientes características físicas:

- **Área y Superficie:** Un espacio de al menos 2×2 metros sobre una superficie plana, rígida y lisa (baldosa o cemento pulido).
- **Materiales de los Obstáculos:** Los muros y el callejón en U estarán construidos exclusivamente con materiales rígidos (madera o cartón). Las paredes formarán ángulos estrictos de 90 grados para garantizar el rebote frontal de las ondas del radar ultrasónico. No se utilizarán telas, espumas ni superficies cilíndricas.

9. Manejo del Repositorio en GitHub

El repositorio sigue la siguiente estructura de ramas para organizar el flujo de trabajo colaborativo:

- **main:** Contiene el código estable y funcional del proyecto.
- **test:** Rama dedicada a las pruebas unitarias. Por cada prueba de un sensor/actuador se realiza un commit; tras su validación, se hace un *merge* a la rama principal.
- **feature/*:** Ramas dedicadas al desarrollo de nuevas integraciones (ej. *feature/gps*, *feature/navegacion_autonoma*).

10. Circuitos Analógicos Requeridos

Dado que los pines de la Raspberry Pi Pico operan a una lógica máxima de 3.3 V, se requiere el diseño de circuitos divisores de tensión:

1. **Divisor de tensión para el Radar:** Para adaptar la señal de salida del pin ECHO (5 V) del HC-SR04 a un nivel seguro de 3.3 V para la RPi Pico.
2. **Divisor de tensión para monitorización de batería:** Como requisito de protección, se implementará un divisor de tensión para reducir los 7.4 V del banco de baterías de los motores a una escala de 0 a 3.3 V, permitiendo al ADC del microcontrolador leer el nivel de carga y activar alarmas preventivas.

11. Repositorio del Proyecto

El código fuente, pruebas unitarias y documentación del proyecto Navix se encuentran disponibles en el siguiente repositorio público de GitHub:

`https://github.com/JuanGonzalez47/navix-autonomous-car`